

[Biometria] Projekt 3 - Dokumentacja

Maciej Andrzejewski, Ludwik Madej

Spis treści

1	Opis projektu	2
1.1	Wykorzystane biblioteki	2
1.2	Struktura projektu	2
1.3	Instrukcja uruchomienia	2
2	Opis metod	3
2.1	Przygotowanie obrazu	3
2.2	Poprawa połączeń - domknięcie morfologiczne	3
2.3	Szkieletyzacja morfologiczna	4
2.4	Algorytm KMM	4
2.5	Algorytm K3M	4
2.6	Detekcja minucji	5
3	Przykłady działania	5
3.1	Przygotowanie obrazu	6
3.2	Porównanie metod ścieniania	6
3.3	Detekcja minucji	7
4	Wnioski	7
4.1	Przebieg implementacji	7
4.2	Porównanie metod ścieniania	8
4.3	Jakość detekcji minucji	8
4.4	Ograniczenia i kierunki rozwoju	8
5	Podsumowanie	8

1 Opis projektu

W ramach projektu zaimplementowano i porównano 3 algorytmy ścieniania w odniesieniu do skanów odcisków palców: szkieletyzację morfologiczną, algorytm KMM oraz algorytm K3M. Zrealizowano również algorytm wyznaczenia lokalizacji minucji (zakończeń linii papilarnych oraz bifurkacji) metodą *Crossing Number*. Całość została przygotowana w języku Python z wykorzystaniem biblioteki NumPy, a wyniki zebrano w notatniku Jupyter.

1.1 Wykorzystane biblioteki

- **NumPy** - do wektorowych operacji macierzowych, stanowi podstawę implementacji wszystkich algorytmów.
- **Matplotlib** - do wizualizacji obrazów, szkieletów oraz nanoszenia punktów minucji na ścieniony obraz odcisku palca.
- **Pillow (PIL)** - wykorzystywana do wczytywania i zapisywania plików.

Algorytmy KMM i K3M zostały zaimplementowane ręcznie na podstawie prac źródłowych K. Saeeda, M. Tabędzkiego, M. Rybnika i M. Adamskiego¹. Operacje morfologiczne zostały zaimportowane z programu stworzonego w ramach Projektu 1.

1.2 Struktura projektu

Kod podzielony jest na pakiet `backend_fingerprints` oraz notatnik porównawczy (testujący) w katalogu `notebooks/`. Poszczególne pliki odpowiadają za następująco:

- `backend_fingerprints/_utils.py` - konwersja *obrazu* $\leftrightarrow$ *maska 0/1*, obsługa konwencji `foreground="dark"/"light"` (czy szukane kontury są jasne, czy ciemne) oraz wektorowe liczenie mapy wag sąsiedztwa 3×3 w zakresie $[0, 255]$
- `backend_fingerprints/_kmm.py` - implementacja algorytmu KMM
- `backend_fingerprints/_k3m.py` - implementacja algorytmu K3M
- `backend_fingerprints/_minutiae.py` - detekcja minucji metodą *Crossing Number*
- `backend_fingerprints/_connect_ridges.py` - domknięcie morfologiczne łączące przerwane linie papilarne
- `notebooks/scienianie_detekcja_minuncji.ipynb` - notatnik, który wczytuje odcisk palca, binaryzuje go, wykonuje domknięcie, uruchamia wszystkie trzy metody ścieniania i nanosi wykryte minucje na każdą wersję szkieletu.

1.3 Instrukcja uruchomienia

Projekt wymaga środowiska Python 3.10 oraz zależności podanych w pliku `requirements.txt`. Zakłada się, że repozytorium zawiera zarówno katalog `Projekt1`, jak i `Projekt3`, ponieważ pakiet `backend` z Projektu 1 jest wykorzystywany bezpośrednio (jego ścieżka dodawana jest do `sys.path`).

1. Przejdź do katalogu projektu:

```
cd Projekt3
```

¹Materiały bazowe znajdują się w katalogu `docs/`: `KMM.pdf` oraz `K3M.pdf`.

2. Utworzyć i aktywować wirtualne środowisko Python 3.10, a następnie zainstalować wymagane zależności:

```
python -m venv .venv
.venv\Scripts\activate      # Windows
source .venv/bin/activate   # Linux / macOS
pip install -r requirements.txt
```

3. Otworzyć notatnik porównawczy:

```
jupyter notebook notebooks/scienianie_detekcja_minuncji.ipynb
```

4. W pierwszej komórce ustawić zmienną `FINGERPRINT_PATH` na ścieżkę do pliku z odciskiem palca w katalogu `data/` oraz, ewentualnie, zmienić `FOREGROUND` ("dark" dla odcisków z ciemnymi liniami na jasnym tle, "light" dla odwrotnej konwencji) oraz próg binaryzacji `BINARIZATION_THRESHOLD`.
5. Uruchomić wszystkie komórki notatnika po kolei.

2 Opis metod

2.1 Przygotowanie obrazu

Wszystkie metody ścieniania operują na obrazie binarnym. Przed ich uruchomieniem obraz jest wczytywany, konwertowany do skali szarości i binaryzowany. Funkcje pomocnicze na obrazach pochodzą z pakietu `backend` z Projektu 1.

Wewnątrz pakietu `backend_fingerprints` wszystkie operacje pracują na jednolitej masce 0/1 typu `uint8`, w której 1 oznacza piksel należący do linii papilarnej, a 0 tło.

Dla KMM i K3M kluczowym elementem jest waga sąsiedztwa. Każdemu z ośmiu sąsiadów w oknie 3×3 przypisano unikalną potęgę dwójki:

$$W = \begin{pmatrix} 128 & 1 & 2 \\ 64 & 0 & 4 \\ 32 & 16 & 8 \end{pmatrix},$$

a wagą piksela jest suma wag tych sąsiadów, którzy są obecni. Wynik mieści się w zakresie $[0, 255]$ i jednoznacznie koduje konfigurację sąsiedztwa, co pozwala na zawarcie wszystkich decyzji (co robić z danym pikselem) w tablicach.

2.2 Poprawa połączeń - domknięcie morfologiczne

Obrazy odcisków palców po binaryzacji często zawierają drobne przerwy wewnątrz linii papilarnych wynikające z niedoskonałości aparatury pomiaru. Przed ścienianiem wykonujemy więc opcjonalnie domknięcie morfologiczne, które częściowo scala poprzerywane fragmenty linii. Robi to funkcja `connect_ridges(image, kernel, foreground)`, która wywołuje odpowiednią funkcję z pakietu `backend` Projektu 1. Domyślnym elementem strukturyzującym jest krzyż 3×3 .

2.3 Szkieletyzacja morfologiczna

Szkieletyzacja morfologiczna używana w tym projekcie to implementacja Lantuéjoula z Projektu 1 (`backend.skeletonize`). Algorytm iteracyjnie eroduje obraz coraz większym elementem strukturyzującym i jako szkielet gromadzi piksele, które znikną po pojedynczej erozji, ale przetrwają otwarcie (tj. piksele znajdujące się „na grzbiecie” obiektu). W każdej iteracji aktualny szkielet to suma kolejnych różnic $E^k(X) \setminus (E^k(X) \circ B)$, gdzie E^k oznacza k -krotną erozję, a $\circ$ otwarcie. Pętla kończy się, gdy obraz całkowicie znika po erozji.

W przeciwieństwie do algorytmów KMM i K3M szkieletyzacja morfologiczna nie ma wbudowanego mechanizmu kontroli ciągłości linii - operuje wyłącznie na lokalnej geometrii elementu strukturyzującego. W efekcie daje szkielet dobrze oddający kształt obiektów, ale podatny na zostawianie drobnych „wąsów” i rozjechanie się linii w obszarach o złożonej topologii.

2.4 Algorytm KMM

KMM jest iteracyjnym algorytmem ścieniania dedykowanym obrazom binarnym. W każdej iteracji wykonywane są następujące kroki:

1. Wszystkie piksele pierwszego planu (krawędzie) otrzymują etykietę 1.
2. Piksele stykające się z tłem krawędzią (N/E/S/W) dostają etykietę 2, a te stykające się z tłem jedynie rogami - etykietę 3.
3. Dla każdego piksela o etykiecie 2 lub 3 sprawdzane jest, czy jego sąsiedzi w pierścieniu 3×3 tworzą jeden spójny łuk o długości 2, 3 albo 4. Sprawdzenie to realizowane jest przez zliczenie zmian tło $\leftrightarrow$ pierwszy-plan w pierścieniu ośmiu sąsiadów w porządku zegarowym - pojedynczy spójny łuk daje dokładnie dwie takie zmiany (wejście w łuk oraz wyjście z niego). Jeśli w istocie tak jest, piksel oznaczany jest etykietą 4 i od razu kasowany.
4. W porządku rastrowym dla każdego piksela o etykiecie 2 liczona jest waga sąsiedztwa zgodnie z macierzą W (por. sekcja 2.1). Jeżeli waga znajduje się w zdefiniowanej przez autorów algorytmu tablicy usunięć, piksel zostaje skasowany; w przeciwnym razie wraca do etykiety 1. Analogiczny krok powtarzany jest dla etykiety 3.
5. Iteracja powtarzana jest tak długo, dopóki w ciągu jednej iteracji usuwany jest choć jeden piksel.

Pełna tablica usunięć znajduje się w pracy źródłowej w dokumencie `docs/KMM.pdf`.

2.5 Algorytm K3M

K3M to uogólnienie i rozszerzenie KMM. Zamiast jednej tablicy usunięć algorytm używa aż sześciu takich tablic ($A_0, A_1, A_2, A_3, A_4, A_5$) oraz dodatkowej tablicy A_{pix} sprowadzającej szkielet do szerokości jednego piksela.

Jedna iteracja K3M dzieli się na sześć faz:

0. Dla każdego piksela pierwszego planu liczona jest waga sąsiedztwa. Wszystkie piksele o wadze z tablicy A_0 są oznaczane jako piksele-brzegi (kandydaci do usunięcia w tej iteracji).
- 1-5. Z kandydatów usuwane są te, których waga znajduje się odpowiednio w tablicy A_1, A_2, A_3, A_4 oraz A_5 . Poszczególne tablice odpowiadają pikselom posiadającym kolejno 3, 3-4, 3-5, 3-6 oraz 3-7 przylegających do siebie sąsiadów. Ponieważ usunięcie piksela zmienia wagę jego sąsiadów, decyzja o skasowaniu podejmowana jest na bieżąco dla aktualnego stanu maski, a nie dla stanu sprzed iteracji (wagi są przeliczane na bieżąco).
6. Pozostałe piksele-brzegi są zostają na obrazie.

Iteracje powtarza się tak długo, dopóki w iteracji zachodzi jakakolwiek zmiana. Po zakończeniu głównej pętli uruchamiana jest dodatkowa faza oparta na tablicy $A_{1\text{pix}}$, która zmniejsza szerokość pozostałych fragmentów szkieletu do dokładnie jednego piksela.

Pełne tablice usunąć $A_0, \dots, A_5$ oraz $A_{1\text{pix}}$ pochodzą bezpośrednio z pracy źródłowej dostępnej w `docs/K3M.pdf`. W implementacji każda tablica jest przechowywana jako 256-elementowa tablica boolowska, dzięki czemu sprawdzenie przynależności wagi staje się pojedynczym odczytem indeksowanym.

2.6 Detekcja minucji

Po wykonaniu ścieniania, na otrzymanym szkielecie szukamy dwóch rodzajów minucji:

- zakończenie linii - piksel szkieletu mający dokładnie jednego sąsiada,
- bifurkacja - piksel szkieletu, z którego wychodzą dwie/trzy osobne linie.

Oba przypadki wykrywamy metodą *Crossing Number*. Dla każdego piksela p w masce 0/1 liczymy:

$$\text{CN}(p) = \frac{1}{2} \sum_{i=1}^8 |P_i - P_{i+1}|,$$

gdzie $P_1, \dots, P_8$ to jego ośmiu sąsiadów w porządku zegarowym, a $P_9 = P_1$. Wartość CN to liczba zmian tło↔pierwszy-plan w pierścieniu 3×3 podzielona przez dwa. Piksele należące do szkieletu rozpoznajemy jako minucje zgodnie z regułą:

- $\text{CN}(p) = 1$ - zakończenie linii,
- $\text{CN}(p) = 3$ - bifurkacja.

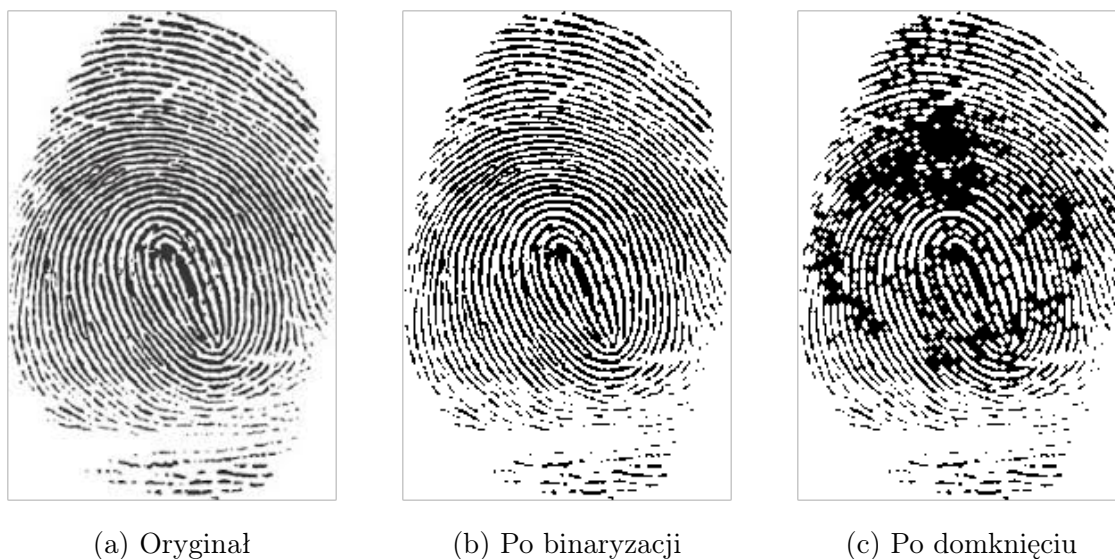
Piksele leżące bliżej krawędzi obrazu niż zadany margines (`border_margin`) są pomijane, ponieważ fałszują detekcję (szkielet bywa poszarpany przez krawędź obrazu, co daje zarówno fałszywe zakończenia, jak i fałszywe bifurkacje).

3 Przykłady działania

Poniższe wyniki uzyskano dla przykładowego odcisku palca² umieszczonego w katalogu `data/`, zbinaryzowanego z progiem 128 i z ciemnymi liniami papilarnymi (`foreground="dark"`).

²Źródło obrazu odcisku palca: Wikipedia

3.1 Przygotowanie obrazu



Rysunek 1: Wstępne przetwarzanie odcisku palca: binaryzacja obrazu wejściowego oraz domknięcie morfologiczne łączące poprzerywane linie papilarne.

Rysunek 1 pokazuje wpływ domknięcia morfologicznego na obraz po binaryzacji. Krzyż 3×3 jako element strukturyzujący pozwala domknąć drobne przerwy w liniach bez znaczącego rozmycia sąsiednich krawędzi.

Z uwagi na dobrą jakość pierwotnego obrazu dalsze operacje zdecydowano przeprowadzać z pominięciem domknięcia morfologicznego.

3.2 Porównanie metod ścieniania



Rysunek 2: Porównanie szkieletów uzyskanych trzema metodami ścieniania.

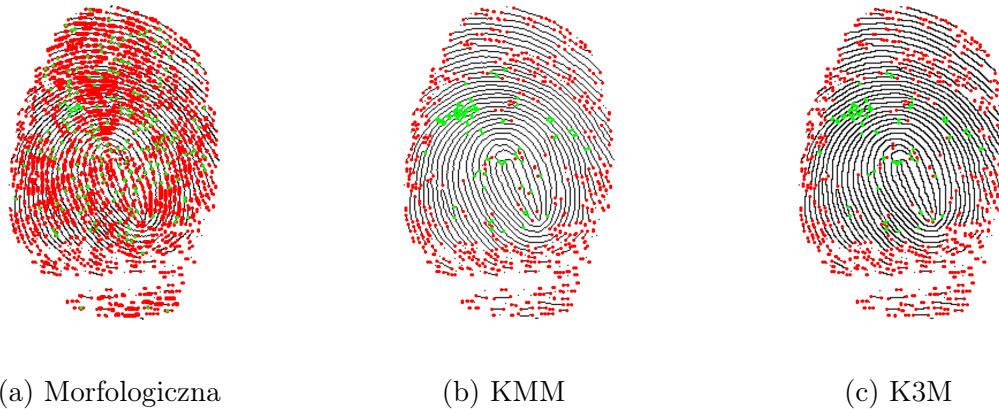
Dla obrazu wejściowego zawierającego 12 890 pikseli pierwszego planu, liczba pikseli pozostałych w szkielecie wygląda następująco:

Metoda	Liczba pikseli szkieletu
Szkieletyzacja morfologiczna	9 513
KMM	6 498
K3M	8 543

Tabela 1: Liczba pikseli pierwszego planu w szkielecie dla poszczególnych metod (oryginalny obraz: 12 890 pikseli).

KMM najmocniej redukuje liczbę pikseli. Szkielet morfologiczny ma aż o połowę więcej pikseli niż szkielet KMM - nie gwarantuje on szerokości jednopikselowej, więc pozostawia fragmenty o szerokości dwóch sąsiednich pikseli w miejscach symetrycznych. K3M jest blisko wyniku KMM, ale zachowuje nieco więcej pikseli, co skutkuje również liczniejszymi zakończeniami w miejscach, gdzie KMM „uciął” krótkie gałęzie.

3.3 Detekcja minucji



Rysunek 3: Minucje wykryte metodą Crossing Number dla każdego ze szkieletów. Kolor czerwony - zakończenia linii (CN=1), zielony - bifurkacje (CN=3).

Metoda	Zakończenia (CN=1)	Bifurkacje (CN=3)
Szkieletyzacja morfologiczna	3 459	201
KMM	594	111
K3M	643	97

Tabela 2: Liczba wykrytych minucji dla każdego ze szkieletów.

Najbardziej widoczna różnica dotyczy metody morfologicznej - wykrywa ona niemal 3 500 zakończeń, podczas gdy KMM i K3M dają ich rzędu 600. Wynika to wprost ze struktury szkieletu morfologicznego, który zawiera liczne wypustki i dwupikselowe fragmenty linii - w każdym takim miejscu CN fałszywie wskazuje zakończenie. KMM i K3M dają wyniki zbliżone, zarówno pod względem liczby zakończeń, jak i bifurkacji.

4 Wnioski

4.1 Przebieg implementacji

Algorytmy KMM i K3M w fazach usuwania pikseli nie są w pełni wektoryzowalne (skasowanie piksela zmienia wagę sąsiedztwa kolejnych kandydatów w tej samej iteracji), dlatego też operacje musiały zostać być zaimplementowane sekwencyjnie, w porządku rastrowym i na bieżącym stanie maski. Biblioteka NumPy wykorzystana jest więc przede wszystkim do szybkiego liczenia mapy wag sąsiedztwa na początku każdej iteracji oraz do stworzenia tablic zamieniające zbiory wag na tablice boolowskie o szybszym dostępie po indeksach.

Najbardziej nieoczywistym elementem okazał się krok algorytmu KMM usuwający piksele posiadające od 2 do 4 przylegających do siebie sąsiadów. Interpretacja tego warunku jako prostego zliczenia sąsiadów daje algorytm, który niszczy ciągłość linii (np. pięciopikselowy prostokąt

redukuje się do dwóch pikseli). Poprawne odczytanie warunku wymaga sprawdzenia, czy sąsiedzi tworzą pojedynczy spójny łuk w pierścieniu 3×3 , co implementujemy licząc liczbę zmian $tlo \leftrightarrow pierwszy-plan$ w porządku zegarowym.

Kod korzysta maksymalnie z pakietu **backend** z Projektu 1 - wczytywanie obrazu, konwersja do skali szarości, binaryzacja, wizualizacja oraz szkieletyzacja morfologiczna są importowane bezpośrednio. Dzięki temu nowy kod dotyczy przede wszystkim nowych algorytmów ścieniania (KMM i K3M) oraz metody detekcji minucji (Crossing Number).

4.2 Porównanie metod ścieniania

Szkieletyzacja morfologiczna jest najprostsza do użycia i dobrze oddaje kształt dużych obiektów, ale nie gwarantuje szkieletu jednopikselowego. W obszarach o złożonej strukturze pozostawia fragmenty o szerokości dwóch pikseli oraz liczne drobne wypustki. Dla odcisków palców, gdzie linie są gęsto upakowane, daje to bardzo zaszumiony obraz i w efekcie ogromną liczbę fałszywych zakończeń przy detekcji minucji.

KMM zaprojektowany jest jako algorytm ścieniania linii papilarnych i widać to w wynikach - daje najbardziej zwarty szkielec, z największym stopniem redukcji liczby pikseli oraz stosunkowo niewielką liczbą fałszywych minucji. Kosztem jest większa złożoność implementacji: etykiety 1/2/3/4, sprawdzanie spójnego łuku sąsiadów oraz konieczność przetwarzania sekwencyjnego.

K3M jest ideowym rozwinięciem KMM - zastępuje etykiety 2/3 i dwuprzebiegowe skanowanie wewnątrz iteracji sześcioma płaskimi fazami usuwania kolejnych „warstw” brzegowych i dodatkową fazą A_{1pix} finalnie sprowadzającą szkielec do szerokości jednego piksela. Otrzymany szkielec jest gładszy niż w KMM (mniej zygzaków), lecz zostawia nieco więcej pikseli.

4.3 Jakość detekcji minucji

Jakość wykrywania minucji zależy wprost od jakości szkieletu - metoda *Crossing Number* jest bardzo prosta i nie koryguje błędów ścieniania. W praktyce oznacza to, że każdy niedoskonały fragment szkieletu daje jedno lub więcej fałszywych zakończeń, a każde zbędne rozdwojenie daje fałszywą bifurkację.

Na naszych danych KMM i K3M dały dużo mniej minucji niż szkielec morfologiczny, co sugeruje znacznie wyższą użyteczność do dalszej obróbki biometrycznej. Między KMM a K3M różnice są niewielkie: K3M daje nieco więcej zakończeń, ale nieco mniej bifurkacji, co jest zgodne z jego mniej agresywnym zachowaniem w sąsiedztwie skrzyżowań linii.

4.4 Ograniczenia i kierunki rozwoju

Największym ograniczeniem obecnej implementacji jest wydajność - KMM i K3M wykonują sekwencyjne pętle po pikselach, co dla dużych obrazów potrafi trwać dużo czasu. Jest to zatem pierwszy potencjalny kierunek rozwoju.

Drugim kierunkiem jest lepsze filtrowanie minucji - obecnie jedynym filtrem jest margines krawędzi. Pełny system biometryczny wymagałby jednak bardziej zaawansowanych operacji.

5 Podsumowanie

Celem ćwiczenia było przeprowadzenie analizy procesu ścieniania obrazów linii papilarnych oraz ocena wpływu zastosowanych metod na jakość detekcji minucji. W ramach pracy porównano trzy podejścia: szkieletyzację morfologiczną oraz algorytmy KMM i K3M. Każda z metod prowadziła do uzyskania jednopikselowego szkieletu, jednak różniły się one jakością odwzorowania struktur charakterystycznych dla odcisków palców.

Przeprowadzone eksperymenty wykazały, że klasyczna szkieletyzacja morfologiczna generowała największą liczbę artefaktów oraz zakłóceń wpływających negatywnie na późniejszą analizę

minucji. Uzyskane szkielety były mniej stabilne i częściej zawierały nieciągłości oraz fałszywe rozwidlenia. Znacznie lepsze rezultaty osiągnięto dla algorytmów KMM oraz K3M, które zachowywały większą spójność linii papilarnych i dokładniej odwzorowywały ich rzeczywisty przebieg.

Algorytm KMM pozwolił uzyskać bardziej zwarty i precyzyjny szkielet, co sprzyjało poprawnej detekcji zakończeń linii oraz bifurkacji. Z kolei metoda K3M generowała gładzsze struktury i lepiej zachowywała ciągłość linii, choć w niektórych przypadkach pozostawiała większą liczbę dodatkowych pikseli. Otrzymane wyniki potwierdzają, że odpowiedni dobór metody ścieniania ma kluczowe znaczenie dla jakości dalszej analizy biometrycznej.

Podsumowując, algorytmy iteracyjne KMM i K3M okazały się znacznie skuteczniejsze od klasycznej szkieletyzacji morfologicznej w kontekście przetwarzania obrazów odcisków palców. Uzyskane rezultaty pokazują również, jak istotnym etapem w systemach identyfikacji biometrycznej jest poprawne przygotowanie obrazu przed ekstrakcją cech charakterystycznych.